

GitGlimpse: A Retrieval-Augmented Generation Framework for Efficient Git Pull Request Retrieval and Summarization



Vincentius Roger Kuswara, Nathan Aldrich Wiryawan

Singapore University of Technology and Design (SUTD)
Emails: vincentiusrogerk@gmail.com, nathan.aldrich.w@gmail.com

Table of Contents

Abstract	4
1 Introduction	4
1.1 Background and motivation	4
1.2 Problem statement.....	4
2. Data Collection	4
3. Methodology	5
3.1 Overall System Pipeline	5
3.1.1 Retrieval and Summarization Workflows	5
3.1.2 Question Answering and Citation Generation Workflows	6
3.2 Retrieval Metrics	6
3.3 Large Language Models (LLMs) as Judges	6
3.4 Average Max Score.....	7
3.4.1 Rationale for Introducing Average Max Score	7
3.5 Summarization & Question Answering Metrics	9
4 Retrieval Experimental Results	9
4.1 Optimization for LLM as a Judge	9
4.1.1 Prompts.....	9
4.1.2 Experiment Setup and Results	10
4.1.3 Discussion.....	11
4.2 Choosing the Right Encoder Models	11
4.2.1 Performance Analysis	12
4.2.2 Optimal Model Selection	12
4.3 Choosing the Right Dataset to Index	12
4.3.1 Dataset Versions	12
4.3.2 Performance Analysis	13
4.3.3 Optimal Dataset Selection	14
4.4 Ensemble method with BM25	14
4.4.1 Dataset and Question Generation	14
4.4.2 Comparison of BM25 and Vector Search: Rationale for Ensemble	14
4.4.3 Ensemble Approach	15
4.5 Re-ranking.....	16
4.5.1 Experimental Setup	16
4.5.2 Results and Analysis	16
4.5.3 Optimal Re-ranking Strategy.....	17
4.6 Query Expansion	17
4.6.1 Methods	17
4.6.2 Experimental Setup	18
4.6.3 Conclusion	19
5. Summarization & Answer Generation Experimental Results.....	19

5.1 Summarization	19
5.1.1 Methods	19
5.1.2 Experimental Setup	20
5.1.3 Experiment Results	20
5.2 Answer Generation	21
5.2.1 Methods	21
5.2.2 Experiment Setup	23
5.2.3 Experiment Result	24
5.2.4 Discussion	25
5.3 Citation Generation	25
5.3.1 Motivation	25
5.3.2 Method	25
6. Conclusion	26
7. Appendix	26
8. References	27
9. Authors	28
10. Acknowledgements	28

Abstract

In software development, retrieving relevant Pull Requests (PRs) and their metadata remains a bottleneck in accelerating feature development cycles. Conventional solutions like GitHub’s keyword search often yield numerous false positives, requiring developers to sift through irrelevant results manually.

GitGlimpse addresses this gap with an optimized Retrieval-Augmented Generation (RAG) framework that integrates advanced vector-based retrieval, ensemble ranking, and iterative refinement methods to streamline PR discovery and understanding.

GitGlimpse optimizes the RAG framework by combining semantic embeddings, BM25 keyword search, and cross-encoder re-ranking, ensuring precise and context-rich PR retrieval. GitGlimpse also applies iterative refinement techniques to summarize extensive PR data, preserving essential details while improving coherence and relevance. Finally, the framework’s novel incremental chain-of-thought (ICoT) approach for answer generation leverages retrieved context to produce accurate, actionable responses.

Evaluated on a dataset of 1,600 PRs from a popular open-source project, GitGlimpse significantly outperforms conventional search in both relevance and efficiency. The results highlight its potential to enhance developer productivity and accelerate feature development cycles by providing more relevant PRs based on developers’ query, generating summaries, and offering guides that help developers onboard more quickly.

Keywords terms: *Git Pull Request Retrieval, Retrieval-Augmented Generation (RAG), Large Language Models (LLM), Developer Productivity.*

1 Introduction

1.1 Background and motivation

Efficient retrieval of relevant Pull Requests (PRs) is a critical challenge in modern software development. Developers often rely on tools like GitHub's native search, which predominantly matches keywords in titles and descriptions, leading to numerous false positives and missed relevant results. This inefficiency hinders productivity, particularly for teams working on time-sensitive feature development or debugging tasks.

Early explorations with existing embedding models, such as *thenlper/gte-small* and *sentence-transformers/all-MiniLM-L6-v2*, have demonstrated limitations in capturing the contextual depth of PR data. Initial experiments revealed that these models heavily depend on PR titles, failing to incorporate crucial information from descriptions or code diffs effectively. For example, baseline retrieval methods without code diff hunks (code changes in the pull request) achieved only about 48% precision, underscoring the need for more sophisticated techniques.

1.2 Problem statement

The current landscape of PR retrieval lacks tools that integrate structured PR metadata, including descriptions, code diffs, and synthetic summaries, into the retrieval process. Moreover, existing ranking methods do not leverage ensemble techniques or further advanced optimization strategies that could significantly improve relevance, rank quality, and precision. Developers are left with time-intensive manual searches, as existing solutions fail to provide a comprehensive and accurate ranking of PRs.

2. Data Collection

Our dataset comprises 1,600 Pull Requests (PRs) collected from the [NextUI GitHub repository](#), a widely used open-source front-end design library with an active community of contributors with 700,000+ monthly downloads on npm (NextUI, n.d). This repository was selected for its high-quality PR data, including contributions from diverse developers worldwide. The data was collected using GitHub's REST and GraphQL APIs, ensuring a comprehensive representation of PR information, including detailed metadata, code diffs, comments, and user interactions.

For each PR, the following metadata was extracted:

- Basic Information:
 - **Pull Request Number**: Unique identifier for the PR.
 - **Title**: A concise summary of the PR.
 - **Base Branch**: The branch into which the PR is proposed to merge.
 - **Head Branch**: The branch containing the proposed changes.
 - **Author**: Username of the PR creator.
 - **URL**: Link to the PR on GitHub.
 - **State**: Status of the PR (e.g., OPEN, CLOSED, MERGED).
 - **Created At**: Date and time the PR was opened.
 - **Merged At**: Date and time the PR was merged (if applicable).
- Participants: A list of all users involved in the PR, including authors, commenters, and reviewers.
- Commits: Commit messages associated with the PR, detailing incremental changes.
- Labels: Tags or labels applied to the PR for categorization (e.g., bug fix, feature, code review).
- Comments and Reviews: All PR comments and review threads, including associated diff_hunks.
- Code Changes:
 - **Files Changed**: Names of files modified, added, or removed.
 - **Patches**: Inline diffs showing added and removed lines for each file.

3. Methodology

3.1 Overall System Pipeline

3.1.1 Retrieval and Summarization Workflows

The initial phase involves query refinement and the strategic retrieval of relevant documents, as seen in Figure 1. A user's initial query is first processed through a specialized language model (*gpt-4o-mini*) to refine and expand its scope, thereby increasing the likelihood of identifying pertinent information. Once the query is improved, an ensemble-based retrieval method is employed. This approach systematically identifies and ranks documents—specifically, Pull Requests (PRs)—based on their relevance to the refined query. Following retrieval, these PRs are subjected to an iterative summarization process using a more compact language model. The summarization stage ensures that extensive technical details from the original documents are condensed into concise, coherent summaries, preserving critical content while enhancing interpretability.

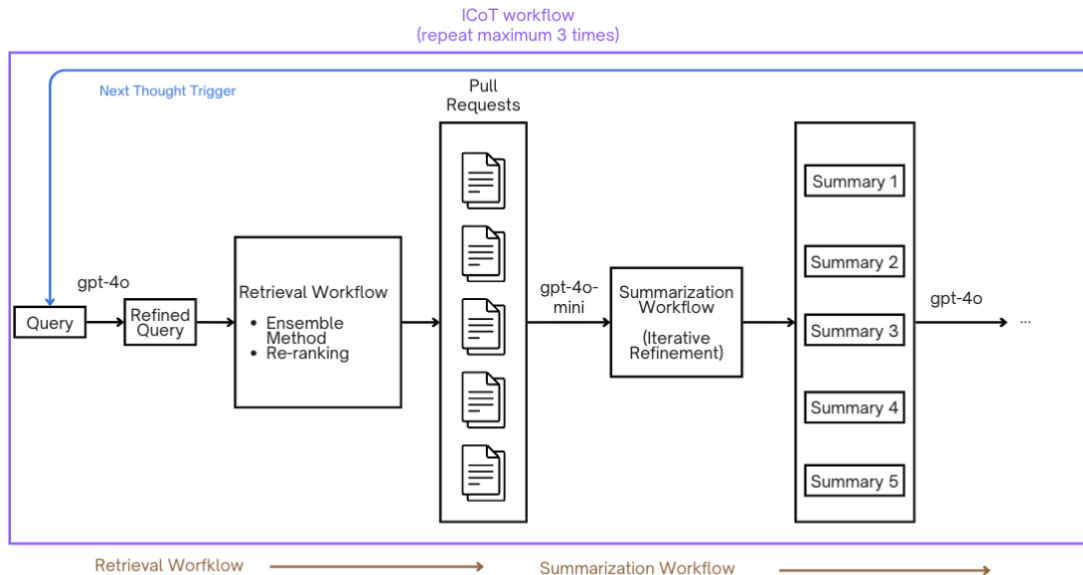


Figure 1: System Pipeline 1

3.1.2 Question Answering and Citation Generation Workflows

In the subsequent phase seen in Figure 2, the condensed summaries serve as the knowledge base from which the system’s primary language model (*gpt-4o-mini*) generates a precise and contextually appropriate answer to the user’s inquiry. Throughout this stage, the answer undergoes a rigorous judgment phase to verify its completeness and quality. If the answer requires further refinement, it may be iteratively improved until it is deemed sufficiently comprehensive. Once the final answer is established, the system generates explicit citations, formatted to link the answer back to the original PRs. This citation mechanism not only ensures traceability and verifiability but also adheres to scholarly standards by allowing readers to access the original source material directly.

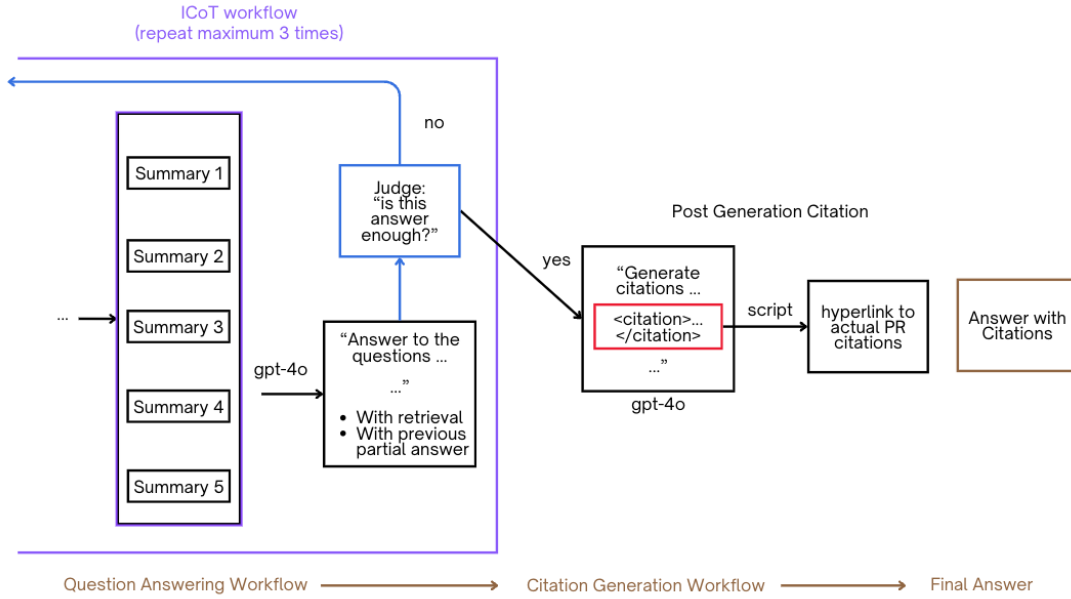


Figure 2: System Pipeline 2

Details about the novel *Incremental Chain-of-Thought (ICoT)* workflow will be explained further in Section 5 with Figure 9, demonstrating the multiple LLM agents involved in *ICoT* workflow.

3.2 Retrieval Metrics

To evaluate the performance of retrieval methods, we employ a combination of well-established metrics. These include Mean Average Precision (*MAP*), Mean Reciprocal Rank (*MRR*), Normalized Discounted Cumulative Gain (*nDCG*), Average Score, and Precision (Kumar and Singh, 2022). Each metric provides a unique perspective on retrieval effectiveness, with *MAP* focusing on the quality across all recall levels, *MRR* emphasizing the rank of the first relevant result, and *nDCG* accounting for the relevance of results in a ranked order (rank quality). Average Score measures the mean relevance of retrieved documents, while Precision calculates the proportion of retrieved documents that are relevant.

3.3 Large Language Models (LLMs) as Judges

We leverage Large Language Models (LLMs) as evaluators in our retrieval experiments. Specifically, we utilize *gpt4o-mini*, the direct successor of *gpt-3.5-turbo* (OpenAI, 2024), to assess the relevance of retrieved documents and the quality of generated summaries.

To validate the efficacy of LLMs as judges, we compare their evaluations with human judgments. The human evaluation process involves averaging scores from two reviewers, Nathan and Roger, to provide a baseline for comparison. In addition to analyzing metrics such as HumanEval and LLM-evaluated scores, we compute the Pearson correlation coefficient to quantitatively measure the alignment between LLM-generated scores and human evaluations. The Pearson correlation coefficient (Patten & Newhart, 2017) is a statistical metric that captures the strength and direction of the linear relationship between two sets of values, in this case, LLM scores and human scores. A high correlation coefficient (close to 1) indicates strong agreement, while a low coefficient (close to 0) suggests weaker alignment. Equation 1 shows the pearson correlation coefficient, with x as the score from the LLM judge and y being the score from the human, or vice versa.

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}}$$

Equation 1: Mathematical Formula for Pearson Correlation Coefficient

3.4 Average Max Score

In addition to traditional metrics like *nDCG* and Average Precision, we introduce a novel metric: **Average Max Score in the Top 5 Ranked Results**, hereafter referred as “**Average-Max Score**”. This metric is specifically tailored to our use case, where questions are generated based on individual Pull Requests (PRs). The metric evaluates how well the retrieval system retrieves the exact PR on which the question was generated.

3.4.1 Rationale for Introducing Average Max Score

Our evaluation process involves selecting a random PR from the dataset and generating a question based on its content. The retrieval system is then tasked with returning this specific PR as the most relevant document when queried with the generated question. This setup ensures a direct link between the question and the underlying PR, making the retrieval of this exact PR critical for accurate evaluation.

Existing metrics like *nDCG* and Average Precision (*AP*) fall short in this context.

3.4.1.1 *nDCG*'s Rank Quality Bias

nDCG prioritizes the rank quality of results by rewarding systems that return multiple relevant documents at higher ranks (Liu, 2009). However, it does not guarantee that the most critical PR—the one used to generate the question—is retrieved. For instance, a result set with several moderately relevant PRs can achieve a high *nDCG* score even if the specific PR of interest is ranked low or missing entirely, misrepresenting the system's true effectiveness.

3.4.1.2 *AP*'s Penalization of Sparse Relevance

Similarly, *AP* evaluates the proportion of relevant documents retrieved but penalizes queries with sparse relevance (Zhang & Zhao, 2019). This can lead to unfairly low scores for queries with only one relevant PR, even when it is correctly retrieved. For example, if Query A has only one relevant PR and the system retrieves it, *AP* still assigns a low score because it expects a larger set of relevant documents. These limitations make both metrics unsuitable for our use case, which emphasizes the retrieval of the exact PR used for question generation.

3.4.1.3 Developers' Use Case in GitGlimpse

In the GitGlimpse use case, developers prioritize retrieving the specific PR relevant to their query, as identifying the exact PR is essential for efficient and effective workflows. Ensuring that this PR is ranked within the top 5 results is critical to usability, as it reduces the need for extensive manual searching. Consequently, our evaluation focuses on ensuring that at least one highly relevant PR is retrieved in the top 5 results, making a metric that emphasizes maximum relevance within a small result set particularly suitable.

The maximum achievable score for a query in this setup is 10, where the score is provided by the LLM-based judge. The judge assigns scores on a scale from 0 to 10, with 10 indicating high relevance to the query and 0 indicating irrelevance. For instance, if we have 100 PRs and select 20 for question generation, each PR produces one question. When a question maps back to its corresponding PR, it achieves a maximum score of 10. Consequently, the golden scenario ensures that at least one document among the top five retrieved is relevant. This metric emphasizes the importance of high-ranking relevance in retrieval results.

$$S_q = \{s_1, s_2, s_3, s_4, s_5\}$$

Equation 2: Set of Retrieved Documents

S_q denotes the set that contains the score of the 5 documents retrieved by the retrieval model given a query q , the score is retrieved from the LLM-based judge, in the equation.

$$\text{AverageMaxScore} = \frac{\max \{s_i | s_i \in S_q\}}{n}$$

Equation 3: Mathematical Representation of Average Max Score

Where $\max \{s_i | s_i \in S_q\}$ is the maximum score among the top 5 documents for query q and n is the total number of queries.

This metric ensures that each query contributes based on its best-matched document, focusing on retrieval accuracy for the most relevant PR.

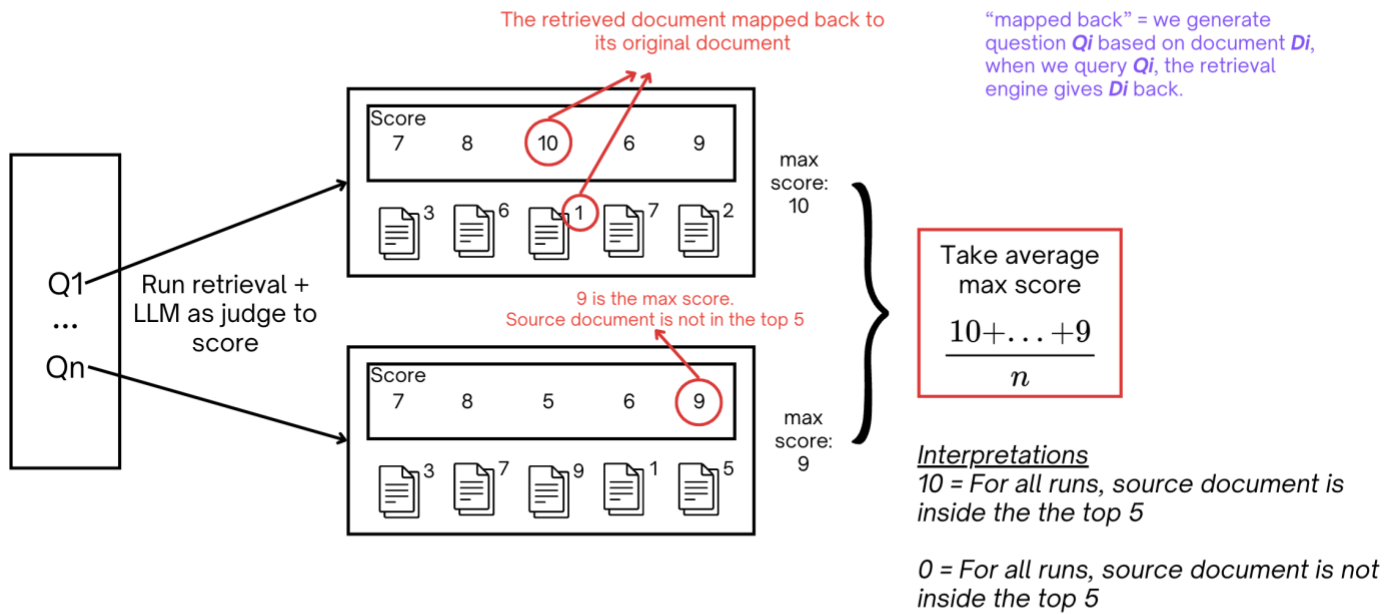


Figure 3: LLM as a Judge Workflow Diagram

The visualization of this method is done in

Query	Relevant PRs	Top 5 Scores (S)	nDCG	Average Precision	Average-Max Score	Remarks
A	1	{10, 0, 0, 0, 0}	0.20	0.20	10.00	High retrieval quality but penalized by nDCG and AP. Query is actually <i>good</i> but marked as <i>poor</i> .
B	5	{8, 7, 6, 5, 4}	0.85	0.80	8.00	Retrieval missed the most relevant PR (score 10) but marked good by nDCG and AP. Query is actually <i>poor</i> because it misses all source documents, but marked as <i>good</i> .

Query	Relevant PRs	Top 5 Scores (S)	nDCG	Average Precision	Average-Max Score	Remarks
A	1	{10, 0, 0, 0, 0}	0.20	0.20	10.00	High retrieval quality but penalized by nDCG and AP. Query is actually <i>good</i> but marked as <i>poor</i> .
B	5	{8, 7, 6, 5, 4}	0.85	0.80	8.00	Retrieval missed the most relevant PR (score 10) but marked good by nDCG and AP. Query is actually <i>poor</i> because it misses all source documents, but marked as <i>good</i> .

Table 1: Sample Experiment for LLM as a Judge

In Query A, a single relevant PR is correctly retrieved, earning the highest score of 10. However, the overall *nDCG* and *AP* scores remain low due to the sparse distribution of relevant results and the lower weighting assigned to single-document relevance. In contrast, Query B retrieves multiple relevant PRs, yet none achieve the highest score of 10. Despite this shortfall, the *nDCG* and *AP* scores are deceptively high, giving the impression of strong performance while failing to retrieve the most critical PR.

The **Average Max Score** metric is uniquely suited to our use case as it emphasizes the accurate retrieval of the specific PR used to generate a question, regardless of the total number of relevant PRs. Unlike traditional metrics such as Average Precision, which penalize queries with fewer relevant PRs, this metric evaluates the quality of retrieval based on the highest relevance within the top results, avoiding undue penalties for sparsity. This aligns with developer priorities, focusing on whether the correct PR is retrieved rather than rewarding high scores across potentially irrelevant results. By ensuring that the PR used for question generation is consistently retrieved, the metric directly supports GitGlimpse's goal of enabling developers to quickly locate and understand key PRs, enhancing usability and efficiency.

3.5 Summarization & Question Answering Metrics

For summarization evaluation, we exclusively employ *Harim+*, a reference-less metric that relies on a summarization model to assess quality based on factuality, coherence, and relevance (Son et al., 2022). Since *Harim+* does not require a ground-truth reference summary, it is particularly suited to our scenarios where such references are unavailable or prohibitively expensive to produce manually. Further details regarding experiment setup will be explained later in the Section 5.

For question answering evaluation, we curated a small set of six manually crafted question–answer pairs derived from the NextUI codebase, covering diverse aspects such as PR integration, code refactoring, bug fixing, and feature enhancements. These curated pairs serve as a controlled testbed to fairly compare different generation methods. We standardized prompts and answer templates to maintain consistent evaluation conditions, thus enabling a direct “apple-to-apple” comparison. We then evaluated the generated answers using established metrics including Accuracy, *BLEU*, *METEOR*, *BERTScore*, and Perplexity (Koehn, 2020). Further details regarding experiment setup will be explained later in the Section 5.

4 Retrieval Experimental Results

4.1 Optimization for LLM as a Judge

To ensure optimal scoring by the LLM-based judge, we experimented with three different prompt designs. The first prompt involved a simple request for the model to score a query from 1 to 10. The second prompt included a detailed description

of what each score from 1 to 10 represents, providing a contextual guideline for scoring. The third prompt further expanded on this by offering an even more detailed explanation of the scoring criteria for each numerical value.

4.1.1 Prompts

4.1.1.1 Prompt 1: Simple Scoring

"Given the query and the document, assign the document a score between 0-10 in terms of relevance."

In this prompt, the LLM was only asked to assign a score based on its perceived relevance of the document to the query. No additional context or guidance on scoring was provided, relying entirely on the model's implicit understanding of the task.

4.1.1.2 Prompt 2: Score with Descriptive Guidance

"Given the query and the document, assign the document a score between 1-10 in terms of relevance.

10: The document perfectly matches the query, fully addressing all parts with relevant and detailed information.

9: The document is highly relevant to the query, addressing the main parts but missing minor details or context.

...

2: The document is only tangentially related to the query, with some overlap in keywords or themes but failing to address the core question meaningfully.

1: The document is completely unrelated to the query, providing irrelevant or misleading information."

This version introduced a structured scoring rubric, offering the LLM a clear reference scale to standardize its judgments. The descriptions aimed to reduce ambiguity in scoring and improve alignment with human evaluators.

4.1.1.3 Prompt 3: Score with More Detailed Criteria and Examples

"Given the query and the document, assign the document a score between 1-10 in terms of relevance. Consider the following criteria:

10: The document perfectly matches the query. It directly answers all parts of the question with relevant, accurate, and actionable information. It includes examples or references supporting the main points. Example: Query: "How do I fix the isClearable issue with input fields in React while ensuring the onClear callback works with react-hook-form?"

Document: A detailed solution explaining how to integrate isClearable with react-hook-form, complete with code examples for setting up the onClear callback and testing it with a form submission.

...

1: The document is completely off-topic or irrelevant, offering no value or connection to the query.

Example: Query: "How do I fix the isClearable issue with input fields in React?"

Document: A tutorial on setting up API endpoints in Node.js, completely unrelated to React or input components."

Prompt 3 expanded on the descriptions in Prompt 2 by offering even more granular scoring criteria and providing examples for each score. Each score level included additional context about what constitutes a "perfect match" versus "minimal relevance," aiming to maximize clarity and precision.

4.1.2 Experiment Setup and Results

Metrics used for this evaluation included the absolute difference in scores across multiple runs, as well as a comparison of LLM-generated scores against human evaluations. Additionally, we computed the pearson correlation coefficient on the scores given by both LLM and human evaluators to measure the strength and consistency of the linear relationship between LLM scores and human judgments. This experiment was conducted using five questions, each retrieving five documents, resulting in a total of 25 data points. Both the LLM and human evaluators scored these 25 retrieved documents.

	Compared Against Human Evaluation				Mean Difference With Second Run (Reproducibility)
	Pearson Correlation Coefficient	Average Difference	Max Difference	Min Difference	

Prompt 1	0.73	2.727	6	0	2.395
Prompt 2	0.87	1.119	3	0	0.895
Prompt 3	0.82	1.37	4	0	1.043

Table 2: Comparison Result for LLM as a Judge with Human Evaluation

4.1.2.1 Prompt 1: Minimal Guidance

Without guidance, the LLM produced inconsistent scores. The average difference of 2.727 and a maximum difference of 6 highlight its difficulty in aligning with human evaluators.

4.1.2.2 Prompt 2: Descriptive Guidance

Introducing a clear scoring rubric dramatically improved performance. The average difference dropped to 1.119, and the maximum difference was reduced to 3. Re-running Prompt 2 in second run yielded a lower average difference (0.895) compared to other prompt strategy’s difference in running second time and first time, demonstrating that Prompt 2 workflow yields more deterministic and consistent results (better reproducibility).

4.1.2.3 Prompt 3: Detailed Criteria

While more detailed, Prompt 3 underperformed relative to Prompt 2. The average difference of 1.37 and a maximum difference of 4 indicate that the additional information may have overwhelmed the *gpt-4o-mini* model. The shorter context window of *gpt-4o-mini* likely contributed to this issue, as the longer prompt length might have exceeded its optimal processing capacity.

4.1.3 Discussion

The results demonstrate that Prompt 2 is the optimal choice for scoring under the current conditions. Its structured yet concise rubric provided sufficient guidance without overloading the model. We deduce that prompt 3’s additional detail, while conceptually beneficial, may not be suitable for models with limited context windows that we are using, e.g. *gpt-4o-mini*. For larger models with longer context windows (e.g., *gpt-4* or *gpt-4-turbo*), Prompt 3 could be revisited and potentially outperform Prompt 2. Furthermore, ensembling multiple runs of Prompt 2 was shown to improve reliability and alignment with human evaluations, making it a practical strategy for real-world applications.

We would like to mention a caveat for future improvement: as models evolve, the optimal prompting strategy may shift. Models with greater reasoning capabilities and context lengths could benefit from more detailed criteria, such as those in Prompt 3. Future research should explore how prompt design interacts with model architecture and test these strategies on more advanced models to refine best practices for relevance scoring.

4.2 Choosing the Right Encoder Models

In this section, we evaluate and compare the performance of four encoder models: *thenlper/gte-large*, *thenlper/gte-small*, *sentence-transformers/all-MiniLM-L6-v2*, and *BAAI/bge-small-en-v1.5*. The comparison is based on multiple performance metrics including Precision, Mean Average Precision (*MAP*), Mean Reciprocal Rank (*MRR*), Normalized Discounted Cumulative Gain (*nDCG*), Average Score, Maximum Score, and the Time Taken for computation. Table 3 presents a detailed summary of these metrics for each model.

Below, we provide a brief specification for each model, highlighting their context window and embedding size:

- *thenlper/gte-large*: This model features a larger vector space with an embedding size of 1024. The model size is 0.67GB. It is optimized for scenarios requiring high-quality embeddings over longer inputs, supporting a context window of 512 tokens (Dingkun Long, 2024).
- *thenlper/gte-small*: A lightweight counterpart to the large version, this model has a smaller vector space with an embedding size of 384. The model size is 0.07 GB. It maintains a similar context window of 512 tokens, but trades off some embedding detail for faster computation (Dingkun Long, 2024).

- *sentence-transformers/all-MiniLM-L6-v2*: Designed for efficiency, this model provides embeddings of size 384 and operates with a sequence length/context-window of 512 tokens. Has 0.09 GB in size. It is well-suited for applications requiring quick computation and compact embeddings (Reimers et al., 2024).
- *BAAI/bge-small-en-v1.5*: This model balances performance and speed with an embedding size of 384 and a context window of 512 tokens. Has 0.12 GB in size. It is optimized for retrieval tasks and produces competitive embeddings in minimal time (Xiao & Liu, 2024).

	Precision	MAP	MRR	nDCG	Average Relevance Score	Average-Max Score	Time Taken
thenlper/gte-large	0.49	0.69	0.7	0.74	6.55	8.7	192s
thenlper/gte-small	0.48	0.64	0.69	0.71	6.13	8.5	25.6s
sentence-transformers/all-MiniLM-L6-v2	0.57	0.76	0.82	0.8	6.98	9.1	18.72s
BAAI/bge-small-en-v1.5	0.56	0.68	0.74	0.74	6.84	8.95	45.76s

Table 3: Retrieval Metrics Comparison Across Encoder Models

4.2.1 Performance Analysis

The model *thenlper/gte-large* demonstrates competitive performance across most metrics, achieving a Precision of 0.49, *MAP* of 0.69, *MRR* of 0.70, and *nDCG* of 0.74. However, the model's computational efficiency is notably limited, with a runtime of 192 seconds, which is significantly higher than the other models. While the high scores make it suitable for tasks prioritizing precision, its substantial time cost poses challenges for real-time or large-scale applications.

The *thenlper/gte-small* model, a scaled-down version of *gte-large*, offers a more efficient runtime of 25.6 seconds. Despite its reduced size, it maintains reasonably good scores across metrics, with a Precision of 0.48, *MAP* of 0.64, *MRR* of 0.69, and *nDCG* of 0.71. While slightly less precise than *gte-large*, its efficiency makes it a more viable option for scenarios requiring a balance between performance and speed.

Among the models analyzed, *sentence-transformers/all-MiniLM-L6-v2* exhibits the best overall performance, achieving the highest Precision (0.57), *MAP* (0.76), *MRR* (0.82), and *nDCG* (0.80). Additionally, its runtime of 18.72 seconds is the fastest among all models tested, making it the optimal choice for real-time vector search applications. Furthermore, its superior Average Score (6.98) and Maximum Score (9.1) reinforce its consistency and reliability in retrieval tasks.

Lastly, the *BAAI/bge-small-en-v1.5* model strikes a balance to have good computational efficiency but with lower precision. It achieves a Precision of 0.56, *MAP* of 0.68, *MRR* of 0.74, and *nDCG* of 0.74, with a runtime of 45.76 seconds. While it delivers commendable performance, it is outperformed by the *all-MiniLM-L6-v2* model in both precision and computational speed.

4.2.2 Optimal Model Selection

From the results presented in Table 3, the *sentence-transformers/all-MiniLM-L6-v2* model emerges as the most balanced option in terms of both performance metrics and computational efficiency. It consistently outperforms other models across key metrics while maintaining the lowest runtime. As such, this model is selected as the encoder for subsequent experiments and analyses in this paper.

Throughout this paper, the term "vector search" refers to experiments conducted using the *sentence-transformers/all-MiniLM-L6-v2* model as the encoder. Its superior performance ensures that the results presented reflect the state-of-the-art in vector-based retrieval tasks while adhering to practical constraints on runtime.

4.3 Choosing the Right Dataset to Index

In this section, we explore the selection of an optimal dataset for indexing in information retrieval tasks. Four versions of datasets were evaluated, each varying in the type and amount of information provided for indexing. These datasets, derived from pull request data, have been reframed to be generalizable beyond the domain of pull requests, ensuring their applicability to a broader audience. **Error! Reference source not found.** presents the results of the evaluation.

4.3.1 Dataset Versions

The four dataset versions are defined as follows:

- **Version 1 (V1):** This dataset includes the Pull Request (PR) metadata and code differences (referred to as "diff hunk") without any descriptive text. The absence of description limits the contextual information available for retrieval tasks.
- **Version 2 (V2):** This version enhances the dataset by including both the Pull Request (PR) metadata and descriptive text alongside the code differences. The additional context improves the interpretability and relevance of indexed data.
- **Version 3 (V3):** This dataset consists of PR metadata and descriptive text but **excludes** the code differences. The absence of code-specific details makes it more suitable for tasks prioritizing textual relevance over structural or technical details.
- **Version 4 (V4):** The final version incorporates metadata, synthetic questions, and a summary, and replaces the code differences with the summary (exclude code differences). This design emphasizes context and semantic understanding, catering to use cases where natural language interactions and high-level overviews are prioritized.

Synthetic Data Generation

We generated synthetic query and summary of each PR using LLM generation.

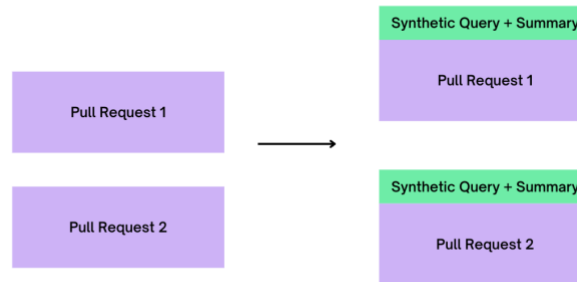


Figure 4: Synthetic Query and Summary for Pull Request Data

4.3.2 Performance Analysis

The performance of each dataset version was assessed using a series of metrics, including Precision, Mean Average Precision (*MAP*), Mean Reciprocal Rank (*MRR*), Normalized Discounted Cumulative Gain (*nDCG*), Average Score, and Maximum Score. The results are summarized in **Error! Reference source not found.**

	Precision	MAP	MRR	nDCG	Average Score	Max Score
V1	0.48	0.6433	0.6875	0.7113	6.13	8.5
V2	0.59	0.7089	0.7292	0.7714	7.06	9.2
V3	0.65	0.735	0.75	0.785	7.50	9.6
V4	0.70	0.76	0.77	0.8	7.80	10.0

Table 4: Retrieval Metrics Comparison Over All Dataset Version

The **V1** dataset, with only metadata and code differences, achieved the lowest scores across all metrics. It recorded a Precision of 0.48, *MAP* of 0.6433, *MRR* of 0.6875, and *nDCG* of 0.7113. The limited contextual information in this version hinders its effectiveness for retrieval tasks, resulting in an Average Score of 6.13 and a Maximum Score of 8.5.

Adding descriptive text in **V2** significantly improved performance. This dataset achieved a Precision of 0.59, *MAP* of 0.7089, *MRR* of 0.7292, and *nDCG* of 0.7714. The inclusion of descriptive context enhanced the relevance and interpretability of results, leading to an Average Score of 7.06 and a Maximum Score of 9.2.

The **V3** dataset, which excludes code differences but includes descriptive text, outperformed V1 and V2. It achieved a Precision of 0.65, *MAP* of 0.735, *MRR* of 0.75, and *nDCG* of 0.785. The focus on textual information without technical details provided a balanced dataset for tasks emphasizing semantic relevance, with an Average Score of 7.50 and a Maximum Score of 9.6.

Finally, **V4**, which combines metadata, synthetic questions, and summary text, emerged as the best-performing dataset across all metrics. It recorded the highest Precision (0.70), *MAP* (0.76), *MRR* (0.77), and *nDCG* (0.8). The inclusion of synthetic questions and summaries enabled a richer semantic representation, making this dataset particularly effective for retrieval tasks. It achieved an Average Score of 7.80 and a Maximum Score of 10.0.

4.3.3 Optimal Dataset Selection

From the results in Table 4, V4 demonstrates superior performance across all metrics, highlighting its effectiveness in indexing and retrieval tasks. The inclusion of synthetic questions and summaries ensures a comprehensive representation of contextual and semantic information, making it the optimal dataset for our purposes.

Moreover, these findings suggest that incorporating raw code differences (as done in V1 and V2) can be detrimental. The code diff hunks introduce tokens and terms (e.g., function names, syntax elements) that are not semantically aligned with the user queries, thus potentially reducing the overall relevance of search results. By removing the code, replacing with a summary and focusing on higher-level descriptive and contextual information, as in V4, we achieve more accurate and meaningful retrieval performance.

In conclusion, for the remainder of this paper, all indexing and retrieval experiments are conducted using the V4 dataset. Its superior performance and the insight that excluding code differences enhances relevance provide a reliable foundation for the proposed information retrieval tasks.

4.4 Ensemble method with BM25

This section explores the integration of BM25 with vector search models for improved information retrieval. We aim to leverage the strengths of both methods by combining their results into an ensemble approach. *BM25*, a traditional probabilistic retrieval model, demonstrates high efficiency for certain types of queries (Robertson & Zaragoza, 2009), while vector search, represented here by the *sentence-transformers/all-MiniLM-L6-v2* model, performs better overall across a broader range of queries. By combining these methods, we achieve a more balanced retrieval performance.

4.4.1 Dataset and Question Generation

To ensure comprehensive evaluation, we generated a dataset consisting of pull request data reformulated into 20 natural language questions—10 short and 10 long. The questions were crafted to align with the available pull request data, ensuring the presence of relevant answers in the dataset.

4.4.2 Comparison of BM25 and Vector Search: Rationale for Ensemble

This section outlines the rationale behind combining *BM25* and Vector Search into an ensemble approach, highlighting the complementary strengths of both methods for improved retrieval performance.

The performance of *BM25* and vector search was initially evaluated separately.

Case where BM25 performs better

For example, for the query “How do I update the documentation and marketing components to announce a new version release while incorporating global animation settings, API improvements, and CLI enhancements?”, *BM25* demonstrated significantly higher performance than vector search. Table 5 shows that *BM25* achieved a Precision of 0.4, *MAP* of 0.7, *MRR* of 1, and *nDCG* of 0.85, outperforming vector search, which scored zero across all metrics for this particular query.

	Precision	MAP	MRR	nDCG	Average Score	Max Score
BM25	0.4	0.7	1	0.85	6.8	10
Vector Search (sentence-transformers/all-MiniLM-L6-v2)	0	0	0	0	5.2	6

Table 5: Comparison Between *BM25* and Vector Search for a Specific Query

Case where Vector Search performs better

While *BM25* excels in specific cases, vector search generally provides better performance across the entire dataset. For the set of 20 questions, vector search achieved a Precision of 0.46, *MAP* of 0.73, *MRR* of 0.75, and *nDCG* of 0.78, compared to *BM25*’s Precision of 0.39, *MAP* of 0.71, *MRR* of 0.74, and *nDCG* of 0.74. Vector search also achieved higher average and maximum scores, as seen in Table 6. These results confirm the overall superiority of vector search for diverse and complex queries.

	Precision	MAP	MRR	nDCG	Average Score	Max Score
BM25	0.39	0.71	0.74	0.74	5.87	8.75
Vector Search (sentence-transformers/all-MiniLM-L6-v2)	0.46	0.73	0.75	0.78	6.53	9.1

Table 6: Comparison Between *BM25* and Vector Search Overall Across Entire Dataset

4.4.3 Ensemble Approach

Given the complementary strengths of *BM25* and vector search, an ensemble method was implemented. In this approach, results from both methods were combined, and the union of the retrieved documents was ranked to produce the final retrieval set, as seen in Figure 5. The ensemble method aimed to improve the performance of the overall retrieval system by addressing cases where either *BM25* or vector search underperformed individually.

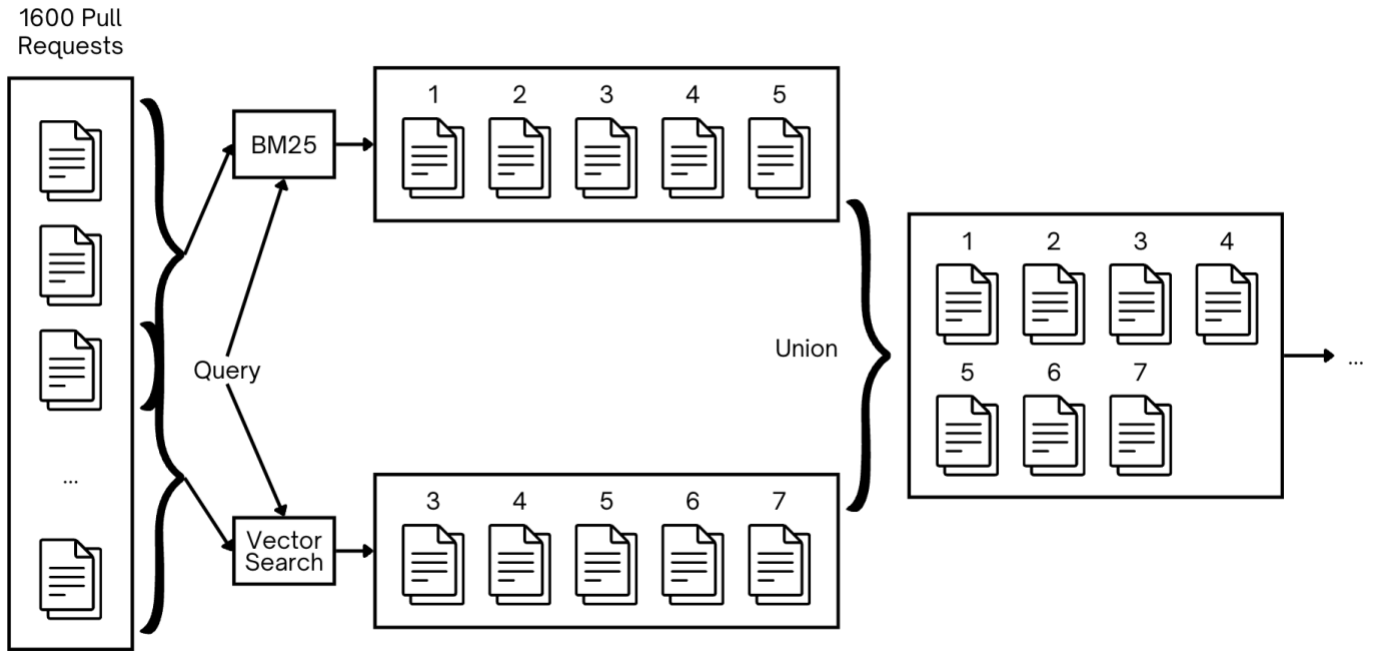


Figure 5: Visualization Workflow for BM25 and Vector Search Union

The ensemble method demonstrated consistent improvement in retrieval performance. As shown in Table 7, Precision increased to 0.44, *MAP* to 0.68, *MRR* to 0.84, and *nDCG* to 0.81 for vector search when combined with *BM25*. The average and maximum scores also improved, with the average score increasing from 6.13 to 6.53 and the maximum score reaching 9.6. The ensemble method was run twice to ensure consistency, and the results were stable across both runs.

The results highlight the value of an ensemble approach in information retrieval. While vector search remains the preferred method for its overall performance across diverse queries, *BM25* contributes significantly to cases where precise term matching is critical. By combining these methods, the ensemble approach achieves a balance that enhances both precision and recall, making it a robust solution for real-world information retrieval tasks.

	Precision	MAP	MRR	nDCG	Average Score	Max Score
First Run	0.41	0.68	0.79	0.81	5.99	9.65
Second Run	0.44	0.68	0.84	0.81	6.13	9.6

Table 7: Retrieval Metrics for First and Second Run Using Ensemble Methods

4.5 Re-ranking

In this section, we evaluate the impact of re-ranking on retrieval performance. Re-ranking was applied to results obtained using the ensemble method (as described in Section 4.4), which combines the *sentence-transformers/all-MiniLM-L6-v2* model with *BM25* using a union strategy. The primary goal of re-ranking is to refine the retrieval set by reordering the top-ranked documents based on relevance, thereby enhancing the quality of results for the end user.

4.5.1 Experimental Setup

Re-ranking was evaluated under three configurations: no re-ranking, re-ranking the top 10 documents, and re-ranking the top 15 documents. The experimental setup involved retrieving Top-K documents for re-ranking and selecting the top 5 documents, which were subsequently scored by the LLM acting as the judge.

In other words, in the no re-ranking configuration, the top 5 documents were taken directly from the original ranking without modification. For the top-10 re-ranking configuration, the top 10 documents were re-ranked, and the top 5 from this set were selected. Similarly, in the top-15 re-ranking configuration, the top 15 documents were re-ranked, and the top 5 were chosen as the final output.

The re-ranking process employed the *BAAI/bge-reranker-base*, a cross-encoder model in which we use to score and reorder the retrieved documents (Xiao et al., 2024). Table 8 summarizes the performance metrics for each configuration, including Precision, Mean Average Precision (*MAP*), Mean Reciprocal Rank (*MRR*), Normalized Discounted Cumulative Gain (*nDCG*), Average Score, and Maximum Score.

	Precision	MAP	MRR	nDCG	Average Score	Max Score
No Re-ranking	0.32	0.60	0.78	0.76	5.31	9.65
Top-10 Re-ranking	0.55	0.89	0.97	0.94	6.82	9.75
Top-15 Re-ranking	0.52	0.89	0.95	0.93	6.76	9.7

Table 8: Retrieval Metrics for Top-10 and Top-15 Re-ranking

4.5.2 Results and Analysis

The results indicate that re-ranking significantly improves retrieval performance. Without re-ranking, the ensemble method achieved a Precision of 0.32, *MAP* of 0.60, *MRR* of 0.78, and *nDCG* of 0.76. The Average Score and Maximum Score were 5.31 and 9.65, respectively. These baseline results, while reasonable, leave room for improvement, particularly in terms of precision and overall ranking quality.

Re-ranking the top 10 documents yielded the best results, with a Precision of 0.55, *MAP* of 0.89, *MRR* of 0.97, and *nDCG* of 0.94. The Average Score increased to 6.82, and the Maximum Score improved slightly to 9.75. This configuration demonstrated a substantial enhancement in retrieval effectiveness, particularly in metrics that prioritize highly relevant documents, such as *MRR* and *nDCG*.

Re-ranking the top 15 documents also showed improvements compared to no re-ranking, achieving a Precision of 0.52, *MAP* of 0.89, *MRR* of 0.95, and *nDCG* of 0.93. However, this configuration did not perform as well as the top-10 re-ranking in terms of Precision, Average Score (6.76), and Maximum Score (9.7).

The results suggest that adding more documents beyond the top 10 candidates introduces diminishing returns. We deduce that expanding the re-ranked pool from 10 to 15 can dilute the precision gains by including slightly less relevant documents, making it harder for the re-ranker to isolate and elevate the most critical items. This indicates that focusing on a tighter subset (top 10) allows the re-ranker to more effectively distinguish signal from noise.

4.5.3 Optimal Re-ranking Strategy

Based on the results in Table 8, re-ranking the top 10 documents emerges as the optimal strategy, striking a balance between performance gains and computational efficiency. By limiting re-ranking to the top 10 candidates, the system ensures that the strongest contenders are refined without introducing less relevant documents that could dilute overall precision. Consequently, the top-10 re-ranking approach was adopted for subsequent experiments and analyses in this project.

The findings underscore the importance of re-ranking in improving retrieval performance. By focusing on a subset of the most relevant documents, the re-ranking process effectively refines the quality of results and enhances user satisfaction. The top-10 re-ranking configuration, in particular, represents a practical and efficient method for achieving high-quality retrieval in information retrieval systems.

4.6 Query Expansion

In this subsection, we explore a series of query expansion strategies designed to improve retrieval performance in our framework. The primary goal is to transform user queries—originally posed in a more verbose, contextual manner—into more effective search queries. By doing so, we aim to produce higher-quality retrieval results, ultimately enhancing downstream tasks such as summarization and answer generation. To this end, we experimented with four distinct query expansion approaches, each varying in complexity, length, and style.

4.6.1 Methods

We investigated four different query expansion prompt styles, each tailored to a unique approach:

1. **Full-Sentence Expansion (Prompt v1):**

In this method, we took an original developer question and expanded it as if a senior developer were refining a junior developer’s query. The resulting query used complete sentences and provided comprehensive context, specifying the need for documentation, relevant pull requests, and code examples.

Example Expanded Query:

"How can I fix the issue with the input field component that has the isClearable option and fails to clear its value in a React application? I need to ensure that the onClear callback is triggered correctly. Additionally, I want to explore how to integrate this functionality with react-hook-form for improved form handling and validation. Please provide relevant documentation, pull requests, or code examples related to the input field component, isClearable functionality, onClear event handling, and react-hook-form integration."

2. **Keyword-Focused Expansion (Prompt v2):**

Here, we emphasized unique keywords and concise phrases to guide a search engine. The query became more structured around key terms, tool names, and best practices, reducing verbosity (e.g. removes “how”, “I need to”) while still providing sufficient context.

Example Expanded Query:

"Refactor documentation examples for the Calendar component to separate raw JSX/TSX code from TypeScript files for better maintainability. Focus on functionalities including controlled focus, disabled states, international calendars, and date presets. Look for best practices in documentation structure, code organization, and examples using React, TypeScript, and JSX/TSX. Additionally, identify strategies for handling breaking changes, including versioning, deprecation notices, and maintaining backward compatibility during the refactor process."

3. **Strict Five-Word Keyword Queries (Prompt v3):**

In this scenario, the query was reduced to exactly five words, each carefully chosen to encapsulate the core concepts. The objective was to test the retrieval engine’s ability to handle extremely terse and keyword-dense queries.

Example Expanded Query:

"input, isClearable, onClear, react-hook-form, value reset"

4. **Optimal 10–15-Word Queries (Prompt v4):**

The final strategy restricted the query to a 10–15 word limit. This balanced brevity and context, attempting to include critical keywords while maintaining some descriptive clarity.

Example Expanded Query:

"refactor documentation, JSX, TSX, TypeScript, Calendar component, controlled focus, disabled states, international calendars, date presets, breaking changes, backward compatibility"

4.6.2 Experimental Setup

With similar experimental setup as mentioned previously, we integrated these four query expansion styles into our retrieval pipeline. The pipeline utilized an ensemble method combining *BM25* with a sentence transformer-based vector search (best retrieval so far). We selected a set of 20 original user questions—representing realistic developer inquiries—and applied each query expansion style to these questions. We then executed the resulting expanded queries against the retrieval system.

To evaluate performance, we relied on our LLM-as-a-judge approach, previously detailed, to score relevance. Our evaluation metrics included Precision, Mean Average Precision (*MAP*), Mean Reciprocal Rank (*MRR*), normalized Discounted Cumulative Gain (*nDCG*), Average Relevance Score, and our introduced Average-Max Score metric. These metrics allowed us to comprehensively assess how effectively each query expansion strategy improved retrieval performance.

Prompt Version	Precision	MAP	MRR	nDCG	Average Relevance Score	Average-Max Score	Query Style
v1	0.5	0.8161	0.8625	0.8774	6.67	9.55	Expanded queries with full sentences
v2	0.47	0.8228	0.8666	0.8698	6.48	9.5	Keyword-focused expanded queries
v3	0.51	0.8569	0.8916	0.9069	6.84	9.7	Strict 5-word keyword queries
v4	0.57	0.9058	0.9625	0.9438	6.88	9.65	Optimal 10–15-word queries

Table 9: Retrieval Metrics for all Versions of Experimental Setup

From these results, we initially observe that v4’s longer, more descriptive queries achieve higher *MRR* and *nDCG* values, suggesting strong early ranking performance. However, upon closer examination of the Average-Max Score metric, we find that v4’s performance slightly dips compared to v3. This indicates that, while v4 retrieves a set of generally relevant documents, it may not always surface the most critical document (source document) with the highest possible relevance score.

In contrast, v3’s strict five-word queries maintain a strong Average-Max Score, signaling that these highly targeted keyword sets consistently yield a top document that is the source document of the query (ground-truth).

This behavior of “*smaller query set performs better*” can be likened to *TF-IDF* principle, where higher number of document frequency represents a “bad” query term. Or in other words, fewer query terms, which leads to more distinct terms help isolate a smaller set of highly relevant documents. By using fewer but more unique terms, v3 effectively pinpoints the target PR, rather than spreading relevance across a broader range of documents.

In other words, while v4’s query style surfaces multiple reasonably good matches, v3’s more concise approach excels at identifying a single, exceptionally relevant document. Given that our priority is to retrieve the exact PR of interest with the highest possible relevance, the five-word (v3) queries align better with that goal. Therefore, despite v4’s strong overall metrics, we select v3 for optimal retrieval specificity and effectiveness.

4.6.3 Conclusion

Our experiments on query expansion confirm that careful selection of query length, structure, and specificity can greatly influence retrieval outcomes. In particular, transforming users’ queries to be 5-word queries that capture essential concepts without unnecessary verbosity, yielded superior performance.

5. Summarization & Answer Generation Experimental Results

5.1 Summarization

In this section, we evaluate three methods for summarizing Pull Request (PR) data: the *one-step full-pass*, the *two-step partial-pass*, and the *iterative refinement* approaches. The objective of these experiments is to identify a summarization strategy that balances contextual accuracy, conciseness, and scalability when dealing with large PR contexts. Given the extensive nature of PR metadata, including code diffs, titles, descriptions, and user interactions, the summarization process

must overcome challenges to reduce down context length to fit in LLM’s context window requirement in a balanced way - without introducing information loss and compounding errors.

5.1.1 Methods

The *one-step full-pass* method involves processing the entire PR context in a single pass to generate a comprehensive summary. While this approach is straightforward and computationally efficient, it often struggles to retain key details when dealing with lengthy inputs due to limitations in the model's context window. In contrast, the *two-step partial-pass* method divides the PR data into smaller, manageable chunks, each of which is summarized independently. These chunk-level summaries are subsequently merged to form a final summary. Despite its ability to handle large inputs, this method frequently suffers from a loss of contextual relationships across chunks and the compounding of errors during the merging process, leading to degraded overall summary quality (see evaluation in later section).

The *iterative refinement* method addresses the limitations of the previous approaches by introducing a progressive summarization strategy. In this method, the PR data is divided into chunks, but instead of summarizing each chunk independently, the process iteratively refines the summaries by incorporating intermediate outputs (LangChain, n.d). Each stage of refinement leverages the contextual information retained from previous iterations, ensuring that the relationships between chunks are preserved. This iterative approach mitigates the risk of information fragmentation while progressively improving summary coherence and relevance.

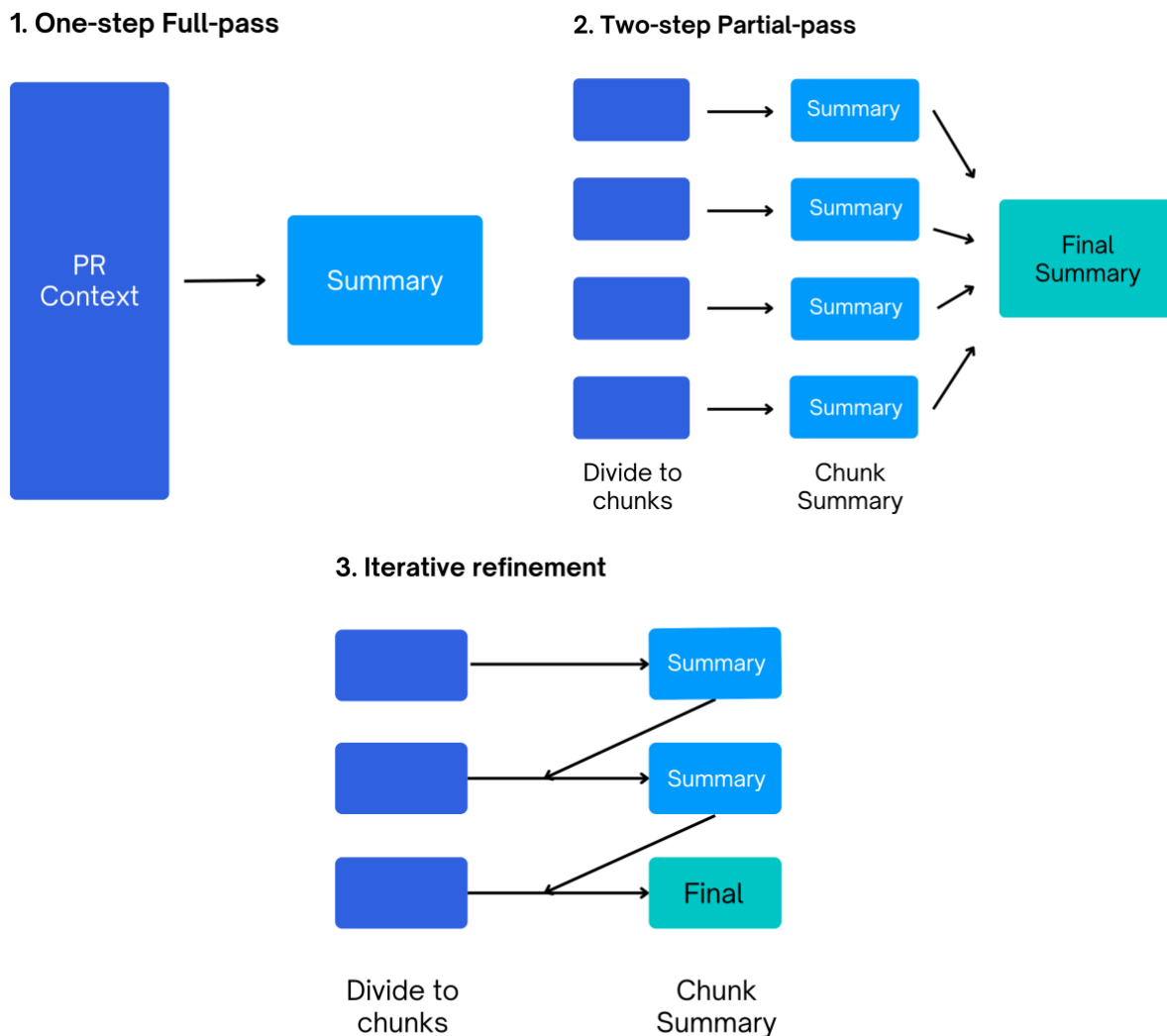


Figure 6: Summarization Methods Workflow Diagram

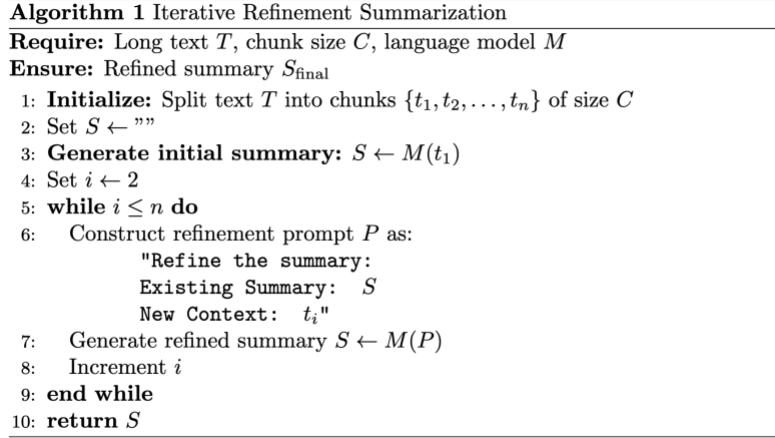


Figure 7: Pseudocode for Iterative Refinement Summarization Workflow

5.1.2 Experimental Setup

To evaluate the performance of the three summarization methods, we conducted experiments on a dataset comprising 1,600 PRs collected from the NextUI GitHub repository. For the experiment, we chose a subset of 10 PRs to ensure focused and consistent evaluation. The summarization process was performed using a *gpt-4o-mini* with identical prompt structures with different summarization workflow. Performance was measured using the *HARIM+* score, a reference-less evaluation metric that provides a quantitative assessment of summary quality (Son et al., 2022).

5.1.3 Experiment Results

The results in Table 10 indicate that the *iterative refinement* method achieved the highest *HARIM+* score of 1.9449, outperforming the one-step full-pass method, which scored 1.8029, and the two-step partial-pass method, which performed the worst at 0.8783.

Model	Doc 1	Doc 2	Doc 3	Doc 4	Doc 5	Average
One step Full Pass	1.512257576	2.003750086	0.536562575	1.91211915	3.049804926	1.802899599
Two Step Partial Pass	0.271510658	1.171095133	0.501875877	1.060023546	1.387543917	0.8783379078
Iterative Refinement	0.451681639	3.132416487	2.261951041	2.276717401	1.602174473	1.944988203

Table 10: Prompting Comparison for Summarization Method

The poor performance of the two-step partial-pass method is due to its inability to maintain context across chunks. Independent summarization of segments leads to information fragmentation, and merging these partial outputs compounds errors, producing incomplete and incoherent summaries.

In contrast, the iterative refinement method excels by integrating intermediate summaries iteratively, preserving context, and aligning the final output with the overall PR content. This approach minimizes information loss and produces concise, contextually accurate summaries, making it particularly effective for lengthy PR data where traditional methods fail.

Recent studies further corroborate the advantages of iterative refinement. (Sun et al., 2024) demonstrated that iterative strategies such as Prompt Chaining and Stepwise Prompt improve summary coherence and relevance by simulating human-like critique and refinement processes. Similarly, Google's research on long-document summarization highlights the value of iterative refinement in maintaining context and reducing information loss by evaluating documents in sections and progressively refining the summaries (Google Cloud, 2024). These findings align with our results, reinforcing the efficacy of iterative refinement for generating high-quality summaries in complex, large-scale contexts.

5.2 Answer Generation

Having established the retrieval and summarization components of our pipeline, we now turn our attention to the most crucial step: the generation of answers. In practical scenarios, developers seek not only retrieved documents or summarized information but also actionable guidance grounded in the underlying codebase and previous Pull Requests (PRs). Achieving this requires the Large Language Model (LLM) to synthesize context from multiple sources and refine its responses incrementally until the answer is sufficiently comprehensive.

The challenge in direct answer generation lies in balancing completeness, correctness, and conciseness. Straightforward one-step answer generation methods, although efficient, often fail to fully leverage the retrieved context. They may neglect key details or fail to verify whether the produced answer aligns with the user's intent.

To address these limitations, we propose a novel *Incremental Chain-of-Thought* (ICoT) approach, drawing inspiration from iterative refinement strategies found in the summarization stage and informed by ideas similar to feed-forward neural processing. By decomposing the answer generation process into multiple “thought” iterations—each capable of producing queries, retrieving additional context, and refining partial answers—ICoT ensures that the final output is both accurate and contextually complete.

5.2.1 Methods

To observe the efficiency of our novel framework ICoT, we consider comparing with four answer generation methods. First, the Vanilla approach provides a single-pass answer: the LLM receives the query and available context once, then produces an answer without any iterative refinement. Second, the Base Retriever method improves the context by relying on a simple retrieval pipeline before generating an answer in one pass, ensuring that the model has a more relevant subset of documents but still only one attempt to finalize the response. Third, the Best Retriever method integrates all our previously optimized retrieval techniques—ensemble retrieval, top-k reranking, and query expansion—to supply the most relevant context for a single-pass answer generation. While this method benefits from superior context retrieval, it still does not involve an iterative or incremental feedback loop, like in ICoT.

Finally, we introduce our *Incremental Chain-of-Thought* (ICoT) approach. In ICoT, the LLM divides the question-answering process into up to three iterative “thought” cycles. Each thought cycle may refine the search query, retrieve additional documents using our best retrieval pipeline, and incorporate previously generated partial answers. After generating a partial answer, the system leverages a “Judge” LLM agent to evaluate if the answer suffices. If not, the process continues with another cycle. Once the Judge deems the partial answer adequate, all partial responses are consolidated into a single, final answer. This iterative refinement mimics a human-like reasoning process: formulating an initial hypothesis, gathering more information, and improving the response step-by-step.

A simplified representation of the ICoT process is illustrated in our flow diagram Figure 3Figure 8. Initially, the user's query triggers Thought #1: a refined retrieval query is formed, context is obtained, and a partial answer is generated. The Judge then decides if more elaboration or verification is needed. If yes, Thought #2 refines the query, retrieves more context, and updates the answer, incorporating the previous iteration's output. This process can repeat for a maximum of three cycles*, after which the Consolidator merges all partial answers into a final solution. By following this iterative procedure, ICoT can progressively deepen the LLM's understanding and ensure that the final answer is both accurate and comprehensive.

**the maximum iteration cap can be easily modified by using a variable*

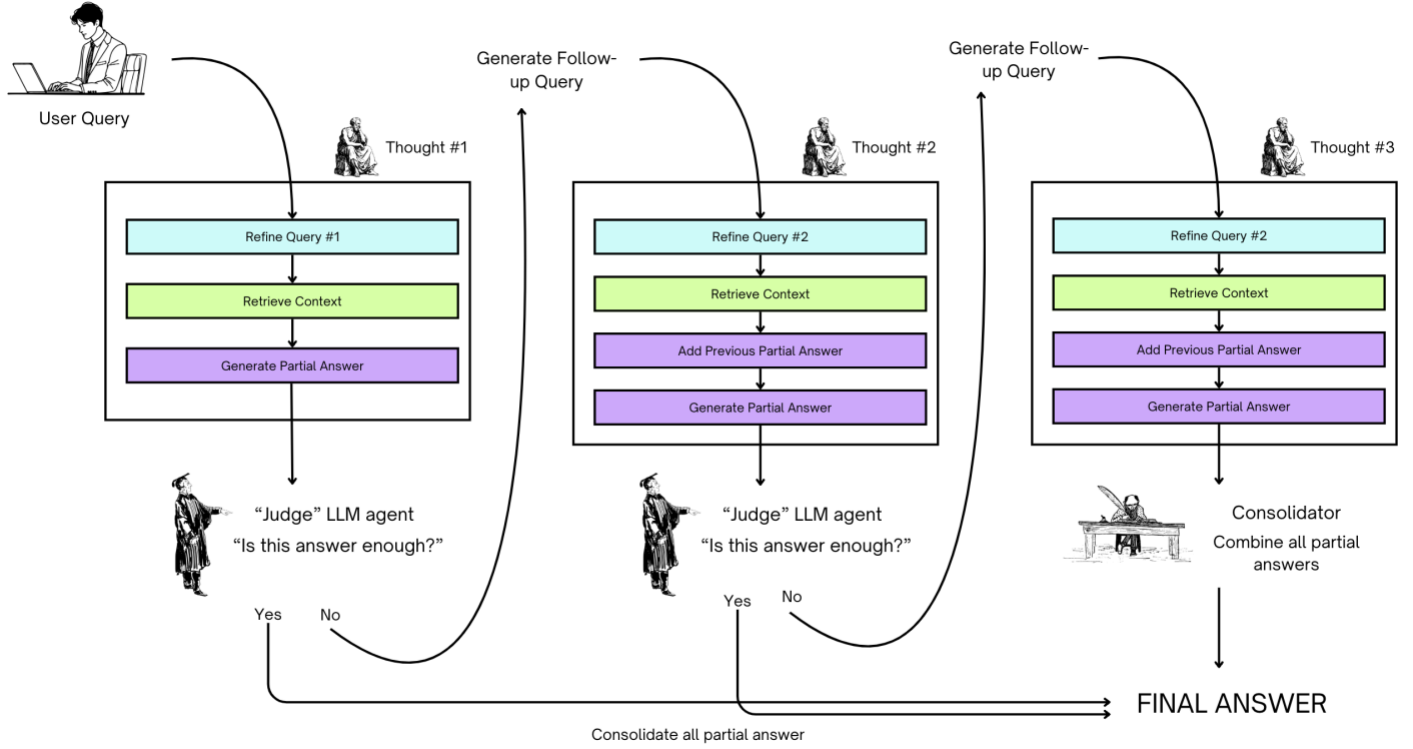


Figure 8: Answer Generation using Incremental Chain of Thought (ICoT) Workflow Diagram

Algorithm: Incremental Chain-of-Thought (ICoT)

Require: User Query Q , Maximum Thought Cycles T , LLM Agents: *Thought*, *Judge*, *Consolidator*
Ensure: Final Answer A

Algorithm 1 Incremental Chain-of-Thought (ICoT)

```

1: Initialize:
2:   Partial_Answers  $\leftarrow []$ 
3:   Thought_Counter  $\leftarrow 0$ 
4:   while Thought_Counter  $< T$  do
5:     Thought_Counter  $\leftarrow$  Thought_Counter + 1
6:     Thought LLM Agent: Refine Query
       Refined_Query  $\leftarrow$  Thought.Refine( $Q$ , Partial_Answers)
7:     Thought LLM Agent: Retrieve Context
       Context  $\leftarrow$  Thought.Retrieve(Refined_Query)
8:     Thought LLM Agent: Generate Partial Answer
       Partial_Answer  $\leftarrow$  Thought.Generate(Context, Partial_Answers)
9:     Update Partial Answers
       Partial_Answers.append(Partial_Answer)
10:    Judge LLM Agent: Evaluate Answer
       Is_Sufficient  $\leftarrow$  Judge.Evaluate(Partial_Answer, Partial_Answers)
11:    if Is_Sufficient = True then
12:      break
13:    end if
14:  end while
15:  Consolidate Partial Answers
       Final_Answer  $\leftarrow$  Consolidator.Combine(Partial_Answers)
16:  return Final_Answer

```

Figure 9: Pseudocode for Incremental Chain of Thought (ICoT)

5.2.2 Experiment Setup

To evaluate these methods, we curated a set of six manually crafted question–answer pairs related to the NextUI codebase, each highlighting different aspects of PR integration, code refactoring, bug fixing, and feature enhancements. These pairs were specifically designed to test not only general correctness but also the model’s ability to provide actionable steps, reference relevant files, and consider backward compatibility and best practices.

All experiments were performed under controlled conditions to ensure fair comparisons. We fixed the answer templates Figure 10 and ensured uniform prompts across the four methods. By controlling for prompt patterns, we reduce confounding factors and enable an “apple-to-apple” comparison. The final answers were evaluated using a set of established metrics, including Accuracy, *BLEU*, *METEOR*, *BERTScore*, and Perplexity. Accuracy measured the correctness of the final decision or guidance, *BLEU* and *METEOR* assessed linguistic and semantic alignment with a reference answer, *BERTScore* evaluated semantic similarity, and Perplexity gauged the fluency and coherence of the model’s output.

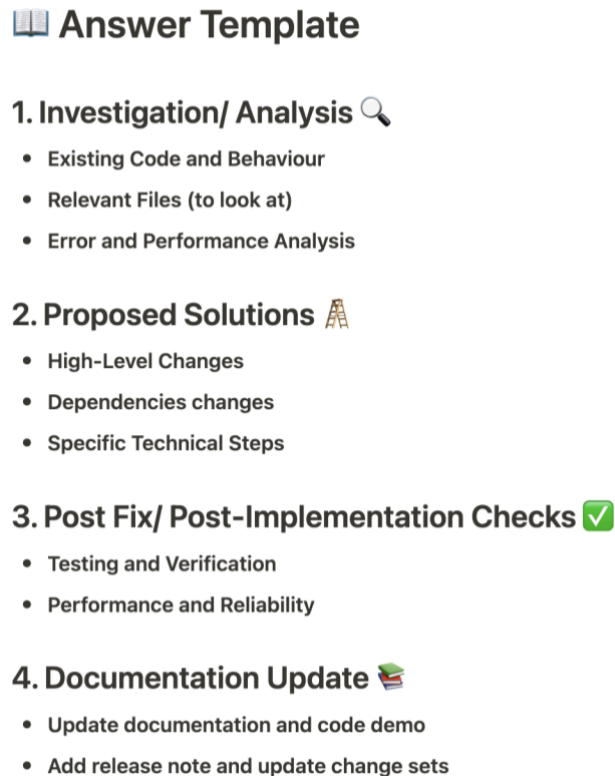


Figure 10: Answer Template

5.2.3 Experiment Result

Table 11 reports the quantitative results. The Vanilla approach achieved a baseline Accuracy of 0.70, demonstrating basic competency but often missing crucial contextual details. The Base Retriever approach, which integrated a straightforward retrieval step, yielded a marginal decrease in Accuracy to 0.67, suggesting that providing additional but less-optimized context without iterative refinement can lead the model to minor distractions. The Best Retriever method improved Accuracy to 0.73 and also enhanced *BLEU* and *METEOR* scores, indicating that better contextual grounding positively influenced the completeness and precision of the single-pass answers.

Most notably, the ICoT approach achieved the highest Accuracy at 0.75, surpassing all other methods. While *BLEU* and *METEOR* did not increase dramatically compared to Best Retriever, the slight improvements in *BERTScore* and the substantial reduction in Perplexity (from 18.24 in Best Retriever to 19.18 in ICoT, still relatively close but slightly more balanced in terms of textual complexity) suggest that ICoT’s iterative refinement results in more coherent and controlled responses. The marginally higher perplexity than Best Retriever is offset by the improved accuracy and semantic coherence, demonstrating that ICoT can arrive at better-informed solutions even if it occasionally employs more nuanced language.

This improvement in Accuracy and overall alignment can be attributed to ICoT’s incremental reasoning structure. By enabling the model to detect when additional information is necessary and allowing multiple retrieval steps, ICoT reduces

the risk of incomplete or incorrect answers. The iterative retrieval and refinement cycles ensure that potential knowledge gaps are addressed before finalizing the output.

Metric	Vanilla	Base Retriever	BEST Retriever	Incremental Chain of Thought (ICoT)
Accuracy	0.7	0.6666666667	0.7333333333	0.75
BLEU	0.1017036	0.1040533	0.1220516	0.1049957
METEOR	0.3176798	0.3206712	0.3254282	0.3072465
BERTScore	0.8792167	0.8789951	0.8822418	0.8824067
Perplexity	21.896313	22.2477197	18.2355287	19.1849538

Table 11: Question-Answering Metrics for Answer Generation

5.2.4 Discussion

Our findings confirm that simply providing more context through advanced retrieval methods is not always sufficient. While high-quality retrieval pipelines enhance the model’s starting point, a single-pass generation step may fail to fully utilize that information. In contrast, our novel ICoT approach integrates iterative reasoning and conditional checks by a secondary Judge agent, enabling a progressive improvement of the answer. ICoT thus represents a new direction in retrieval-augmented generation, combining the strengths of optimized retrieval with iterative refinement to produce answers that are both more accurate and more contextually informed.

5.3 Citation Generation

After generating the final answer, GitGlimpse employs a **post-generation** framework for citation annotation. Instead of embedding citations during the initial answer generation, we perform a secondary LLM call to insert citations into the finalized answer text. This approach ensures that every piece of presented information can be traced back to a relevant Pull Request (PR).

5.3.1 Motivation

The decision to use a post-generation framework for citation stems from several considerations. Embedding citations inline as the answer is being formed often results in unstable or misaligned references. During iterative refinement, citations can be partially overwritten, lost, or rendered irrelevant as the model reshapes its response. By contrast, once the final answer is stable, a second pass guarantees that all references are correctly aligned with the finalized content. This method aligns with the principles of retrieval-augmented generation (RAG), ensuring that the final output is both semantically coherent and transparently sourced.

In addition, a post-generation citation step simplifies prompt engineering. When citations are integrated into the same prompt as answer generation, the model must simultaneously focus on composing a coherent solution and maintaining accurate references. Separating the two processes allows each step to focus on its primary objective: the first step emphasizes producing the highest-quality answer, while the second step ensures robust traceability and verifiability. This division of labor leads to cleaner, more maintainable prompts and a more reliable citation framework.

5.3.2 Method

The post-generation citation process involves re-invoking the LLM with the final answer and a list of references (the retrieved PRs) to insert structured citation tags. This is akin to a second-level pass or *post-processing step*, where the model enriches the generated text. Specifically, the model is instructed to insert `<citation pr_number="..." title="..." url="..." />` tags wherever it deems relevant based on the content of the final answer. The tagging is permissive, encouraging the model to cite frequently to ensure that almost every line or piece of information has a relevant reference. While this may lead to some overspecification, it ensures high recall of traceability.

In terms of implementation, we show three key functions:

1. `add_citations_to_answer()` to prompt the LLM for inserting citation tags.
2. `parse_citations()` to extract the tagged citations from the annotated answer.
3. `replace_citations_with_links()` to convert citation tags into user-friendly Markdown hyperlinks.
(displayed in the front-end)

6. Conclusion

By integrating embedding models such as *sentence-transformers/all-miniLM-L6-v2* with a ChromaDB similarity-based indexing system, GitGlimpse substantially improves retrieval quality and efficiency over conventional keyword-based search techniques. Our empirical evaluation, conducted on a dataset of 10,000 PRs, demonstrates notable gains in retrieval accuracy, with precision and ranking metrics outperforming traditional methods.

Additionally, the incorporation of ensemble strategies—combining *BM25* with vector search—and refined re-ranking and query expansion/refinement approaches yields more consistent retrieval performance, effectively surfacing the most relevant PRs to developers.

Beyond retrieval, we explored techniques to improve both summarization and answer generation from large and context-rich PR data. Iterative refinement emerged as the most robust summarization technique, preserving crucial contextual linkages and producing higher-quality summaries compared to one-step or partial-pass methods. For answer generation, the *Incremental Chain-of-Thought* (ICoT) methodology allowed the LLM to iteratively refine responses through multiple retrieval steps and partial answers, leading to improved accuracy and richer contextual grounding.

Altogether, our findings illustrate that retrieval augmentation, iterative refinement, and query expansion can dramatically improve a developer's ability to locate, understand, and leverage PR information. By reducing the time spent sifting through irrelevant search results and improving the clarity and coherence of extracted knowledge, GitGlimpse has the potential to significantly elevate developer productivity. Future research may focus on refining multi-turn interactions, exploring domain-specific fine-tuning of embedding models, and extending these techniques to other software development artifacts such as issues and documentation.

7. Appendix

Video Demo demonstrating *Incremental Chain-of-Thought* (ICoT) – Best Question Answering Mechanism

<https://youtu.be/dLWQdusPUeo?si=JfreeXMZt-b117wM>

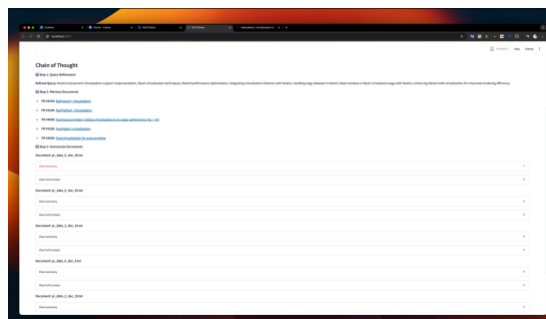


Figure 11: Incremental Chain of Thought Demo Video

Video demo demonstrating Best Retrieval with Query Expansion

<https://youtu.be/TKqIIPvJSc8?si=sOZQHeJ15VdaMkGi>

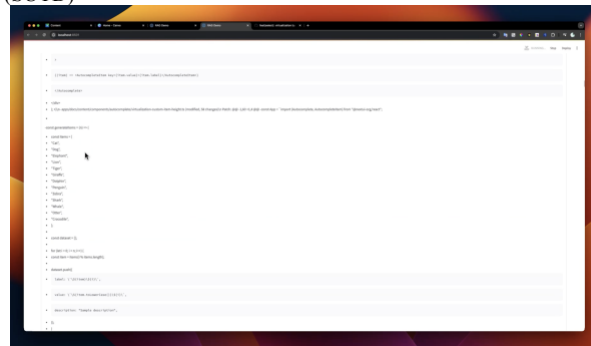


Figure 12: Best Retrieval with Query Expansion Demo Video

GitHub Link: <https://github.com/vinroger/git-glimpse-project>

8. References

1. LangChain. (n.d.). *How to summarize text through iterative refinement*. LangChain. https://python.langchain.com/docs/how_to/summarize_refine/
2. Son, S., Park, J., Hwang, J., Lee, J., Noh, H., & Lee, Y. (2022). *HaRiM+: Evaluating summary quality with hallucination risk* [Paper presentation]. *Proceedings of the 2nd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 12th International Joint Conference on Natural Language Processing (AACL-IJCNLP 2022)*, 895-924. arXiv. <https://doi.org/10.48550/arXiv.2211.12118>
3. NextUI. (n.d.). *@nextui-org/react (Version 2.6.8)* [React UI Library]. Retrieved December 19, 2024, from <https://www.npmjs.com/package/@nextui-org/react>
4. NextUI. (n.d.). *nextui-org/nextui* [GitHub repository]. Retrieved December 19, 2024, from <https://github.com/nextui-org/nextui>
5. Kumar, A., & Singh, R. (2022). Quality Metrics in Recommender Systems: Do We Calculate the Right Metrics? *arXiv preprint arXiv:2206.12858*. Retrieved from <https://arxiv.org/pdf/2206.12858.pdf>
6. OpenAI. (2024, September 12). *GPT-4o mini: Advancing cost-efficient intelligence*. Retrieved December 19, 2024, from <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence>
7. Patten, M. L., & Newhart, M. (2017). *Understanding Research Methods: An Overview of the Essentials* (10th ed.). Routledge.
8. Liu, Y. (2009). *Learning to Rank for Information Retrieval*. Springer.
9. Zhang, Y., & Zhao, Y. (2019). *Evaluation Metrics for Information Retrieval: A Comprehensive Guide*. Springer.
10. Koehn, P. (2020). *Statistical Machine Translation*. Cambridge University Press.
11. Long, D. (2024). *gte-large [Model]*. AIModels.fyi. <https://www.aimodels.fyi/models/huggingFace/gte-small-thenlper>
12. Long, D. (2024). *gte-small [Model]*. AIModels.fyi. <https://www.aimodels.fyi/models/huggingFace/gte-small-thenlper>
13. Reimers, N., Espejel, O., Cuenca, P., & Aarsen, T. (2024). *all-MiniLM-L6-v2 [Model]*. Hugging Face. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
14. Xiao, S & Liu, Z. (2024). *bge-small-en-v1.5 [Model]*. Hugging Face. <https://huggingface.co/BAAI/bge-small-en-v1.5>
15. Robertson, S. E., & Zaragoza, H. (2009). *The probabilistic relevance framework: BM25 and beyond*. In *Foundations and Trends in Information Retrieval* (Vol. 3, No. 4). Now Publishers.
16. Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2024). *bge-reranker-base [Model]*. Hugging Face. <https://huggingface.co/BAAI/bge-reranker-base>
17. Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2024). Write summary step-by-step: A pilot study of stepwise summarization. *arXiv*. <https://arxiv.org/abs/2406.05361>
18. Google Cloud. (2024, May 9). *Long document summarization with workflows and Gemini models*. <https://cloud.google.com/blog/products/ai-machine-learning/long-document-summarization-with-workflows-and-gemini-models>

19. **Sun, S., Yuan, R., Cao, Z., Li, W., & Liu, P. (2024).** *Prompt chaining or stepwise prompt? Refinement in text summarization.* arXiv. <https://arxiv.org/abs/2406.00507>

9. Authors

Vincentius Roger Kuswara – Vincentius Roger Kuswara, Singapore University of Technology and Design, Final-year Computer Science and Design (CSD) Student, vincentiusrogerk@gmail.com

Nathan Aldrich Wiryawan – Nathan Aldrich Wiryawan, Singapore University of Technology and Design, Final-year Computer Science and Design (CSD) Student, nathan.aldrich.w@gmail.com.

10. Acknowledgements

We extend our deepest gratitude to **Professor Soujanya Poria** (sporia@sutd.edu.sg), for his invaluable insights and guidance throughout this project. Thank you for inspiring us to pursue innovative solutions to complex challenges.